

Wksh 2_2 : Recursion

Examples:

Pseudo code example	Equivalent in C language (With C++ facilities)								
Algo Exo_1_01_a Var X: Integer Function Fact(N : Integer) : Integer Begin If N = 0 then Return(1) Else Return(N*Fact(N-1)) EndIf End Begin Read (X) Write(Fact(X)) End	<pre>#include <iostream> using namespace std; int X; int Fact(int N) { if(N == 0) return(1); else return(N*Fact(N-1)); } int main() { cin >> X; cout << Fact(X); }</pre> <table border="1"> <thead> <tr> <th>X</th><th>Screen</th></tr> </thead> <tbody> <tr> <td>0</td><td>1</td></tr> <tr> <td>3</td><td>6</td></tr> <tr> <td>5</td><td>120</td></tr> </tbody> </table>	X	Screen	0	1	3	6	5	120
X	Screen								
0	1								
3	6								
5	120								
	<pre>include <iostream> using namespace std; int X; int Fact(int N) { int R; cout<<"Call with "<< N << endl; if(N == 0) R = 1; else R = N*Fact(N-1); cout << "End Call with " << N << " : " << R << endl; return(R); } int main() { cin >> X; cout << Fact(X); }</pre> <table border="1"> <thead> <tr> <th>X</th><th>Screen</th></tr> </thead> <tbody> <tr> <td>5</td><td> Call with 5 Call with 4 Call with 3 Call with 2 Call with 1 Call with 0 End call with 0: 1 End call with 1: 1 End call with 2: 2 End call with 3: 6 End call with 4: 24 End call with 5: 120 120 </td></tr> </tbody> </table>	X	Screen	5	Call with 5 Call with 4 Call with 3 Call with 2 Call with 1 Call with 0 End call with 0: 1 End call with 1: 1 End call with 2: 2 End call with 3: 6 End call with 4: 24 End call with 5: 120 120				
X	Screen								
5	Call with 5 Call with 4 Call with 3 Call with 2 Call with 1 Call with 0 End call with 0: 1 End call with 1: 1 End call with 2: 2 End call with 3: 6 End call with 4: 24 End call with 5: 120 120								

Exercise 1.18:

Algo Exo1_18_b

```

Var
  X, Y : Integer
Function Power(B, P : Integer):Integer
Begin
  /* Write("Call with ", B, " ", P) */
  If P = 1 then
    Return(B)
  Else
    Return(B * Power (B, P-1))
  EndIf
End
Begin
  Read(X,Y)
  Write(Power(X,Y))
End

```

X	Y	Screen
2	3	8
3	5	243

Algo Exo1_18_c

```

Var
  X, Y : Integer
Function GCD(A, B : Integer) :Integer
Begin
  /* Write("Call with ", A, " ", B) */
  If A = B then
    Return(A)
  Else
    If A > B then
      Return(GCD(A-B,B))
    Else
      Return(GCD(A,B-A))
    EndIf
  EndIf
End
Begin
  Read(X,Y)
  Write(GCD(X,Y))
End

```

X	Y	Screen
21	15	3
38	95	19
4	32	4
7	7	7

Exercise 1.19 (Home work):

Algo Exo1_19_a

```

Var
  X : Integer
Function Bin(N : Integer) : Integer
Begin
  If N = 0 then
    Return(0)
  Else
    Return(Bin(N div 2)*10 + N mod 2)
  EndIf
End
Begin
  Read(X)
  Write(Bin(X))
End

```

N	Screen
0	0
1	1
2	10
13	1101
43	101011

Algo Exo1_19_b

```

Var
  X : Integer
Function Bin(N: Integer):Integer
Var
  R : Integer
Begin
  Write("Call with ", N)
  If N = 0 then
    R ← 0
  Else
    R ← Bin (N div 2)*10 + N mod 2
  EndIf
  Write("End call with ", N, ":", R)
  Return(R)
End
Begin
  Read(X)
  Write(Bin(X))
End

```

N	Screen
43	Call with 43 Call with 21 Call with 10 Call with 5 Call with 2 Call with 1 Call with 0 End call with 0: 0 End call with 1: 1 End call with 2: 10 End call with 5: 101 End call with 10: 1010 End call with 21: 10101 End call with 43: 101011 101011

Examples (Home work):

<u>Pseudo code example</u>	<u>Equivalent in C language</u> <u>(With C++ facilities)</u>																		
Algo Exo_1_05_a Const D = 5 Type TAB = Array of D Real Var Tb : TAB Function SumArr(T : TAB, i_bg: Integer) :Real Begin If i_bg = D-1 then Return(T[D-1]) Else Return(T[i_bg] + SumArr(T, i_bg + 1)) EndIf End Begin For i ← 0 to D-1 do Read(Tb[i]) EndFor Write(SumArr(Tb,0)) End	<pre>#include <iostream> using namespace std; const int D =5; typedef float TAB[D]; TAB Tb; float SumArr(TAB T, int i_bg) { if(i_bg == D-1) return(T[D-1]); else return(T[i_bg] + SumArr(T, i_bg+1)); } int main() { int i; for(i=0;i<D;i++) cin>> Tb[i]; cout << SumArr(Tb, 0); }</pre> <table><tr><th colspan="5">Tb</th><th>Screen</th></tr><tr><td>1</td><td>3</td><td>2.5</td><td>4</td><td>3</td><td>13.5</td></tr><tr><td>2</td><td>5.25</td><td>12</td><td>7</td><td>1</td><td>27.25</td></tr></table>	Tb					Screen	1	3	2.5	4	3	13.5	2	5.25	12	7	1	27.25
Tb					Screen														
1	3	2.5	4	3	13.5														
2	5.25	12	7	1	27.25														

Exercise 1.22_c & 1.22_d (Home work) :

Algo Exo_1_22_c

Const

D = 5

Type

TAB = Array of D Real

Var

Tb : TAB

VI : Real

Function Nb_Occ(T : TAB, Val : Real, i_bg: Integer) : Integer

Var

Co : Integer

Begin

If i_bg < D **then**

Co ← Nb_Occ(T, Val, i_bg + 1)

If T[i_bg] = Val **then**

Return(1 + Co)

Else

Return(Co)

EndIf

Else

Return(0)

EndIf

End

Begin

For i ← 0 **to** D-1 **do**

Read(Tb[i])

EndFor

Read(VI)

Write(Nb_Occ(Tb,VI,0))

End

Tb	VI	Screen
1 3 2.5 4 3	3	2
2 5.25 12 7 1	8	0

Algo Exo_1_22_d

Const

D = 5

Type

TAB = Array of D Real

Var

Tb : TAB

VI : Real

Function Find(T : TAB, Val : Real, i_bg: Integer) : Boolean

Begin

If i_bg < D **then**

If T[i_bg] = Val **then**

Return(True)

Else

Return(Find(T, Val, i_bg + 1))

EndIf

Else

Return(False)

EndIf

End

Begin

For i ← 0 **to** D-1 **do**

Read(Tb[i])

EndFor

Read(VI)

Write(Find(Tb,VI,0))

End

Tb	VI	Screen
1 3 2.5 4 3	3	True
2 5.25 12 7 1	8	False

Exercise 1.21 (Additional):

Function U(n : Integer) : Integer Begin If n = 0 or n = 1 then Return(n) Else Return(U(n-1) + U(n-2)) Endif End	Function V(n : Integer) : Real; Function U(n : Integer) : Real Begin If n = 1 then Return(0) Else Return(2*U(n-1) + V(n-1)) Endif End Function V(n : Integer) : Real Begin If n = 1 then Return(4) Else Return(V(n-1) + U(n-1)/5) Endif End
---	--

Exercise 1.20 (Additional):

Function Remainder(A, B : Integer) : Integer Begin If A < B then Return(A) Else Return(Remainder(A – B, B)) Endif End	Procedure Rmdr(A, B : Integer ; REF R : Integer) Begin If A < B then R ← A Else Rmdr(A – B, B, R) Endif End
Function Quotient(A, B : Integer) : Integer Begin If A < B then Return(0) Else Return(Quotient(A – B, B) + 1) Endif End	Procedure Quotient2(A, B : Integer ; REF Q : Integer) Var Qt : Integer Begin If A < B then Q ← 0 Else Quotient2 (A – B, B, Qt) Q ← Qt + 1 Endif End
Function Division(A, B : Integer ; REF R : Integer) : Integer Begin If A < B then R ← A Return(0) Else Return(Division (A – B, B, R) + 1) Endif End	Procedure Dvs(A, B : Integer ; REF Q, R : Integer) Var Qt : Integer Begin If A < B then Q ← 0 R ← A Else Dvs (A – B, B, Qt, R) Q ← Qt + 1 Endif End

Exercise 1.22_b (Additional):

<pre> Const D = 100 Type TAB = Array of D Real Function MinArr(T : TAB, i_bg: Integer) : Real Var M : Real Begin If i_bg = D-1 then Return(T[D-1]) Else M ← MinArr(T, i_bg + 1) If M < T[i_bg] then Return(M) Else Return(T[i_bg]) EndIf EndIf End /* First call : Mn ← MinArr(TB, 0) */ </pre>	<pre> Const D = 100 Type TAB = Array of D Real Function MinArr2(T : TAB, i_end: Integer) : Real Var M : Real Begin If i_end = 0 then Return(T[0]) Else M ← MinArr2(T, i_end - 1) If M < T[i_end] then Return(M) Else Return(T[i_end]) EndIf EndIf End /* First call : Mx ← MinArr2(TB, D-1) */ </pre>
<pre> Const D = 100 Type TAB = Array of D Real Function MinArr3(T : TAB, i_bg, i_end: Integer) : Real Var i_ml : Integer M1, M2 : Real Begin If i_bg = i_end then Return(T[i_bg]) Else i_ml ← (i_bg + i_end) div 2 M1 ← MinArr3(T, i_bg, i_ml) M2 ← MinArr3(T, i_ml+1, i_end) If M1 < M2 then Return(M1) Else Return(M2) EndIf EndIf End /* First call : S ← MinArr3(TB, 0, D-1) */ </pre>	